

Projeto MICROC – Etapa 3

Parser LL(1) por descida recursiva e construção da AST

2º semestre de 2026

Objetivo

Nesta etapa, o grupo implementará o parser da MICROC por descida recursiva. O parser recebe a sequência produzida pelo lexer do próprio grupo, reconhece a gramática LL(1) fornecida e constrói diretamente uma árvore sintática abstrata.

Resultado da etapa

fonte MICROC → tokens → parser LL(1) → objeto Program

A AST será reutilizada na análise semântica e na geração direta de assembly RISC-V 64. Por isso, os nós concretos, seus campos, os intervalos de fonte e o campo genérico de metadados já fazem parte da interface desta etapa.

1 Material inicial e entrega

O starter contém:

- `source_grammar.ebnf`, com a gramática LL(1) completa;
- `ast_nodes.py`, com todos os nós, enums, spans e serialização;
- `ast_printer.py`, com visualizações em árvore e Graphviz DOT;
- `parser.py`, com a infraestrutura e os métodos a completar;
- `Lexer.py`, contendo somente a interface da etapa anterior;
- `runner.py` e `test.mc`, para executar o pipeline;
- `tests/test_parser.py`, com testes públicos, e `tests/cases/`, com programas `.mc` válidos e inválidos; e
- README, configuração do `pytest` e workflow do GitHub Actions.

Antes de começar, substitua `Lexer.py` pela implementação entregue pelo grupo na etapa do lexer. Não copie `grammar.py`: o projeto de transformação de gramáticas é independente, e a gramática LL(1) necessária já é fornecida.

É permitido criar funções auxiliares privadas em `parser.py`. Não altere nomes, campos ou assinaturas públicas dos tokens, da AST, de `Parser` ou de `ParserError`. A entrega será feita integralmente pelo repositório do grupo no GitHub Classroom.

Prazo desta etapa

[PREENCHER DATA E HORA ANTES DA PUBLICAÇÃO]

2 A gramática fornecida

O sufixo `.ebnf` indica que o arquivo usa *Extended Backus–Naur Form* (Forma de Backus–Naur estendida). A extensão compacta padrões que, em BNF, exigiriam não terminais auxiliares e produções vazias.

Notação	Significado	Equivalência
$A \mid B$	alternativa A ou alternativa B	$A \mid B$
$X?$	zero ou uma ocorrência de X	$X \mid \varepsilon$
X^*	zero ou mais ocorrências de X	repetição com base ε
X^+	uma ou mais ocorrências de X	$X X^*$
$(\dots)$	agrupa uma sequência ou alternativas	um único átomo da notação

Símbolos consecutivos são concatenados. Os operadores pós-fixos `?`, `*` e `+` aplicam-se ao átomo imediatamente anterior; parênteses alteram esse agrupamento. `|` separa alternativas.

```
parameter_list ::= parameter (COMMA parameter)*
return_statement ::= KW_RETURN expression? SEMICOLON
arguments ::= (expression (COMMA expression)*)?
```

Nomes em maiúsculas são categorias de `TokenKind`; nomes em minúsculas são não terminais. EOF é um terminal e deve ser consumido explicitamente. O arquivo é uma especificação para leitura humana: não implemente um leitor de EBNF e não carregue a gramática durante a execução.

2.1 Relação com o código

Cada não terminal possui um método `parse_*`. Uma alternativa é escolhida somente pelo token corrente. Use condicionais para `?`, laços para `*`/`+` e chamadas de métodos para não terminais. O parser não deve voltar o cursor nem tentar alternativas por tentativa e erro.

Nas expressões binárias, cada nível chama o nível de maior precedência e usa um laço para produzir associação à esquerda. Assim, `10 - 3 - 2` deve formar `(10 - 3) - 2`. Operadores unários usam recursão e associam à direita.

3 Interface do parser

A entrada pública é:

```
program: Program = Parser(tokens).parse()
```

`tokens` deve ser uma sequência não vazia, terminar em exatamente um EOF e não possuir outro EOF. O parser conserva um índice para o token corrente. `peek`, `check`, `advance`, `match` e `expect` já estão implementados.

Também são fornecidos:

- `parse_program`, que implementa `function* EOF`;
- `parse_function`, que demonstra sequência e parte opcional; e
- `parse_type`, um auxiliar simples para os três tokens de tipo.

Os demais métodos devem consumir exatamente a construção descrita por seu não terminal e retornar o nó, lista ou valor indicado na assinatura. Um método não deve consumir o primeiro token da construção seguinte.

4 Árvore sintática abstrata

Todos os nós derivam de `Node` e possuem:

```
span: SourceSpan
metadata: dict[str, object]
```

`SourceSpan` registra linha e coluna inicial inclusivas e linha e coluna final exclusivas. `metadata` começa como um dicionário novo e vazio em cada nó. Ele será preenchido por etapas futuras e não deve ser usado para guardar informação necessária ao parser.

- **Programa e funções:** `Program`, `FunctionDecl` e `Parameter`.
- **Blocos e comandos:** `Block`, `VarDecl`, `Assignment` e `CallStmt`.
`IfStmt`, `WhileStmt`, `ReturnStmt` e `PrintStmt` completam o grupo.
- **Expressões:** `BinaryExpr`, `UnaryExpr` e `CallExpr`.
`IdentifierExpr`, `IntLiteral` e `BoolLiteral` completam o grupo.
- **Impressão:** uma expressão ou `StringLiteral` implementa `PrintItem`.

A AST usa enums próprios, não tokens: `TypeName` para tipos; `UnaryOperator` para operadores unários; e `BinaryOperator` para operadores binários. Parênteses não criam um nó: eles alteram a precedência e ampliam o span da expressão interna.

4.1 Campos concretos

Além de span e metadata, os nós possuem estes campos:

```

Program(functions)
FunctionDecl(return_type, name, parameters, body)
Parameter(type, name)
Block(statements)

VarDecl(type, name, initializer)
Assignment(target, value)
CallStmt(call)
IfStmt(condition, then_block, else_block)
WhileStmt(condition, body)
ReturnStmt(value)
PrintStmt(items)

BinaryExpr(operator, left, right)
UnaryExpr(operator, operand)
CallExpr(name, arguments)
IdentifierExpr(name)
IntLiteral(value)
BoolLiteral(value)
StringLiteral(value)

```

`initializer`, `else_block` e o `value` de `ReturnStmt` podem ser `None`. Todas as coleções preservam a ordem do programa.

O alvo de `Assignment` é um `IdentifierExpr`. Uma chamada usada como comando forma `CallStmt(call=CallExpr(...))`; usada como valor, o `CallExpr` aparece diretamente na expressão. Literais de string não são expressões gerais.

4.2 Strings adjacentes em print

A produção `string_literals ::= STRING_LITERAL+` deve concatenar os `value` decodificados de todos os tokens adjacentes e construir um único `StringLiteral`. Assim:

```
print("resultado " "final = ", x);
```

possui dois itens: um `StringLiteral("resultado final = ")` e a expressão `x`.

5 Limite entre sintaxe e semântica

O parser reconhece formas. A análise semântica posterior resolverá nomes, validará tipos, escopos, inicialização, assinaturas, retornos e `main`. Portanto, o parser deve construir AST para exemplos como:

```

void f(void x) {
    void y = true;
    if (1) { return 2 + false; }
}

```

```
}
```

Essas formas possuem erros semânticos, mas obedecem à gramática. Do mesmo modo, arquivo vazio, funções duplicadas e programa sem `main` são aceitos pelo parser para diagnóstico na fase apropriada.

São erros sintáticos, entre outros, `print()`, `string` fora de `print`, expressão isolada como comando, comando vazio, atribuição encadeada, declaração múltipla e ausência das chaves exigidas por `if` ou `while`.

6 Erros sintáticos

O parser interrompe no primeiro erro e lança `ParserError`. A exceção possui os atributos públicos `token`, `expected`, `line` e `column`. O diagnóstico textual informa a posição, o conjunto esperado e o token encontrado.

```
erro sintático em 3:1: esperado {SEMICOLON}, encontrado RIGHT_BRACE  
('}')
```

Não é exigida recuperação, sincronização nem acumulação de erros. O runner escreve o diagnóstico em `stderr` e termina com status 1. Erros de uso ou leitura terminam com status 2.

7 Restrições

O parser deve ser implementado manualmente por descida recursiva, com um token de lookahead. Não é permitido usar `SLY`, `PLY`, `Lark`, `ANTLR` ou outro gerador de parser. Também não é permitido:

- substituir o parser por expressões regulares sobre o programa;
- implementar backtracking ou procurar uma derivação por tentativa;
- executar análise semântica durante o parsing;
- alterar a EBNF para aceitar uma linguagem diferente; ou
- substituir os objetos da AST por dicionários ou pelo JSON do runner.

JSON, árvore textual e DOT são apenas visualizações. O resultado da etapa é o objeto `Program` e seus descendentes.

8 Execução e testes

Depois de copiar o lexer do grupo:

```
python -m pip install -r requirements-dev.txt  
python runner.py --format tree test.mc  
python runner.py --format dot test.mc > ast.dot  
dot -Tsvg ast.dot -o ast.svg  
python -m pytest -q
```

O formato `tree` mostra recursivamente nós, campos e índices das listas no terminal. O formato `dot` pode ser convertido pelo `Graphviz` para SVG ou PDF. A opção

`-show-spans` inclui as posições do fonte em ambos; o formato `json` permanece disponível para inspeção automatizada.

Os testes públicos cobrem a interface e exemplos centrais. Os programas usados ficam em `tests/cases/valid` e `tests/cases/invalid`; seus nomes indicam a construção ou o erro exercitado e cada arquivo pode ser passado isoladamente ao runner. Testes privados adicionais exercitarão todas as alternativas da gramática, precedência, associatividade, spans, erros e casos de borda. Eles não acrescentam regras não publicadas neste documento.

9 Checklist de entrega

Antes do prazo, verifique se:

- o `Lexer.py` do grupo substituiu o esqueleto e todos os métodos `parse_*` foram implementados;
- cada construção consome seus próprios tokens e listas preservam a ordem;
- associatividade, strings adjacentes e spans seguem este enunciado;
- regras semânticas não foram antecipadas e `pytest` passa; e
- o último commit aparece no repositório correto e no GitHub Actions.